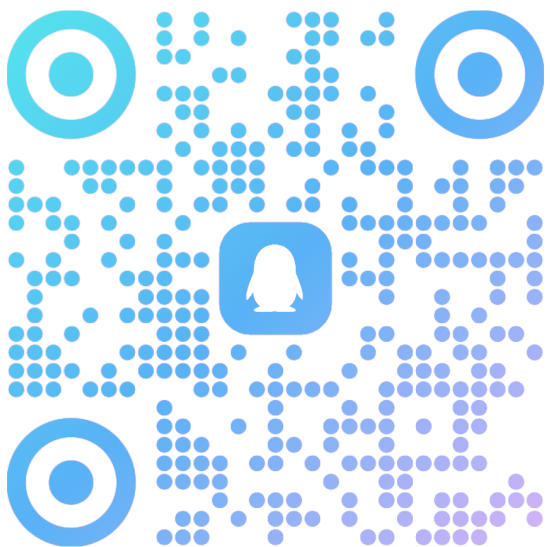




福州大学-2025编译...

群号: 1037351079



扫一扫二维码，加入群聊



编译原理

2025 春

任课教师: 柯嵩宇

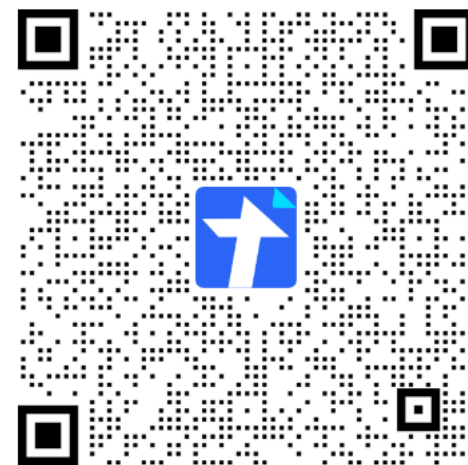
Email: songyuke@fzu.edu.cn

2025-03-06



课堂签到打卡

扫一扫二维码打开或分享给好友



— 腾讯文档 —

可多人实时在线编辑，权限安全可控

课前复习

汇编第二次作业_102201538_黄志梁 ☆



为什么臭着脸 <zl_03@qq.com>

收件人 我

附件 2个 ·  汇编第二次作业_102201538_黄志梁.docx...

 2个附件 | 保存到 [文件云盘](#) 方便管理  [全部保存](#)



汇编第二次作业_102201538_黄志梁.docx (18KB)



汇编_102201538_黄志梁.docx (816KB)



快捷回复给 为什么臭着脸

四类文法

Grammar ⇄	Languages ⇄	Recognizing automaton ⇄	Production rules (constraints) ^[a] ⇄	Examples ^{[5][6]} ⇄
Type-3	Regular	Finite-state automaton	$A \rightarrow a$ $A \rightarrow aB$ (right regular) or $A \rightarrow a$ $A \rightarrow Ba$ (left regular)	$L = \{a^n n > 0\}$
Type-2	Context-free	Non-deterministic pushdown automaton	$A \rightarrow \alpha$	$L = \{a^n b^n n > 0\}$
Type-1	Context-sensitive	Linear-bounded non-deterministic Turing machine	$\alpha A \beta \rightarrow \alpha \gamma \beta$	$L = \{a^n b^n c^n n > 0\}$
Type-0	Recursively enumerable	Turing machine	$\gamma \rightarrow \alpha$ (γ non-empty)	$L = \{w w \text{ describes a terminating Turing machine}\}$

绝大多数编程语言的语法都是上下文无关文法，但如果要考虑语义信息的话，它就变成了上下文相关语言了

正则语言

- 令 r 为一个正则表达式, 则 $L(r)$ 为其所定义的正则语言

[illegible]

例题

- 写一个01字符的正则表达式，其所定义的语言不包含“100”子串

$$0^*1^*(01^+)^*0?$$

$0^*1^*(01^+)^*0?$ 对应的NFA?

$0^*1^*(01^+)^*0?$ 对应的NFA?

$0^*1^*(01^+)^*0?$ 对应的NFA?

$0^*1^*(01^+)^*0?$ 对应的NFA?

$0^*1^*(01^+)^*0?$ 对应的NFA?

状态	0	1	
0			1
1	2		3
2			1
3		4	5
4			3
5	6		9
6		7	
7		8	5
8			7
9	10		11
10			11

$0^*1^*(01^+)^*0?$ 对应的NFA?

状态	0	1	
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	5, 9, 11
8			7, 5, 9, 11
9	10		11
10			11

$0^*1^*(01^+)^*0?$ 对应的DFA?

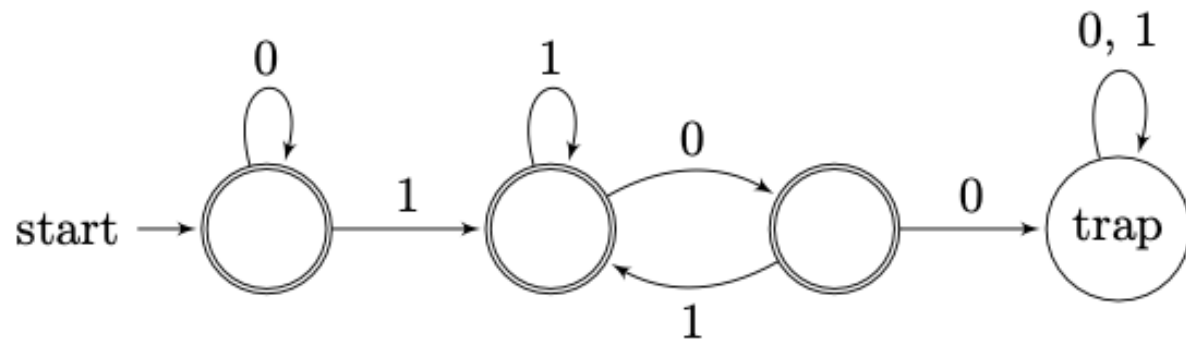
编号	可状态集合	0	
A (起始)	{0, 1, 3, 5, 9, 11}	{2, 1, 3, 5, 9, 11, 6, 10}	{4, 3, 5, 9, 11}
B	{2, 1, 3, 5, 9, 11, 6, 10}	{2, 1, 3, 5, 9, 11, 6, 10}	{4, 3, 5, 9, 11, 7}
C	{4, 3, 5, 9, 11}	{6, 10, 11}	{4, 3, 5, 9, 11}
D	{4, 3, 5, 9, 11, 7}	{6, 10, 11}	{4, 3, 5, 9, 11, 8, 7}
E	{6, 10, 11}	{}	{7, 5, 9, 11}
F	{4, 3, 5, 9, 11, 8, 7}	{6, 10, 11}	{4, 3, 5, 9, 11, 8, 7}
G	{7, 5, 9, 11}	{6, 10, 11}	{8, 7, 5, 9, 11}
H	{8, 7, 5, 9, 11}	{6, 10, 11}	{8, 7, 5, 9, 11}
J	{}	{}	{}

$0^*1^*(01^+)^*0?$ 对应的DFA?

编号	可能状态集合	0	
A (起始)	{0, 1, 3, 5, 9, 11}	{2, 1, 3, 5, 9, 11, 6, 10}	{4, 3, 5, 9, 11}
B	{2, 1, 3, 5, 9, 11, 6, 10}	{2, 1, 3, 5, 9, 11, 6, 10}	{4, 3, 5, 9, 11, 7}
C	{4, 3, 5, 9, 11}	{6, 10, 11}	{4, 3, 5, 9, 11}
D	{4, 3, 5, 9, 11, 7}	{6, 10, 11}	{4, 3, 5, 9, 11, 8, 7}
E	{6, 10, 11}	{}	{7, 5, 9, 11}
F	{4, 3, 5, 9, 11, 8, 7}	{6, 10, 11}	{4, 3, 5, 9, 11, 8, 7}
G	{7, 5, 9, 11}	{6, 10, 11}	{8, 7, 5, 9, 11}
H	{8, 7, 5, 9, 11}	{6, 10, 11}	{8, 7, 5, 9, 11}
J	{}	{}	{}

$0^*1^*(01^+)^*0?$ 对应的DFA?

$0^*1^*(01^+)^*0?$ 对应的DFA?



词法分析器

词法分析的任务（词法分析是什么）

- 词法分析的任务
 - 逐行扫描源程序，识别出各个单词，确定单词的类型。同时将识别出的单词转换成统一的机内表示——词法单元(简称词元，英文：token)形式。

词法分析器的作用

- 读入源程序，去除注释、空白字符（空格、制表符“\t”、换行符“\n\r”等）
 - 中文空格算空白字符吗？
- 建立符号表
 - 一个“键-值”映射表，一般用HashMap或者TreeMap实现（对应C++的map和unordered_map）
 - 去除原始变量名（可能很长）
- 识别词素，抽象成记号
 - 如<id, 1>, <keyword, if>, <op, +>
 - 处理字符串、浮点数、整数字面值等
- 发现词法错误，建立错误消息与源程序的位置联系
 - 例如，0d1234（非法标识符），D"Hello World"（非法的字符串前缀）等
- 将记号提交给语法分析器

思考

词法分析是语法分析的前置步骤

语法分析的理论基础是CFG，词法分析的基础是RG

而CFG是包含RG的，因此能不能直接基于字符进行语法分析？

词法分析器的输出

- 词法分析器的输出：
 - <单词种别, 单词自身的值>
- 程序语言单词的种类：
- 关键字(保留字或基本字)：如 while, if
- 标识符：用来表示各种名字，如变量名、数组名
- 常数：各种类型的常数 256, 3.14
- 运算符：+、-、*、/、等
- 分界符：分号

词法分析器的输出

- 单词种别通常用整数编码表示
 - 符号
 - 一符一码
 - 关键字
 - 可分成一类
 - 也可以一个关键字分成一类（一字一码）。
 - 常数
 - 可统归一类
 - 也可按类型（整型、实型、布尔型等），每个类型的常数划分成一类（一类型一码）。
- 所有的标识符分为一类。
 - 自定义类型名字
 - 变量名
 - 函数名
 - 模板名字

词法分析器的输出

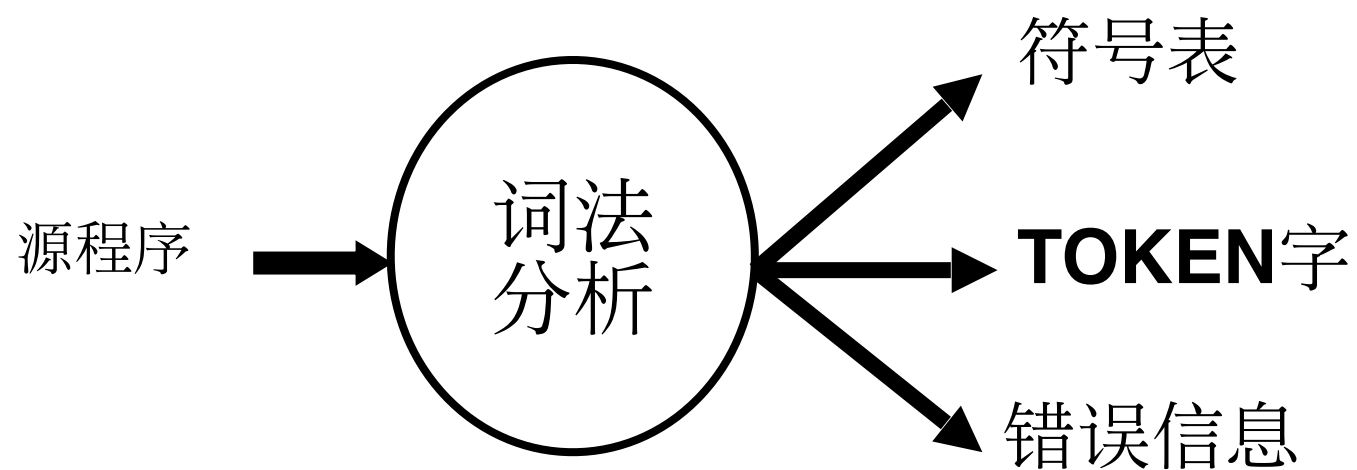
- 单词的属性值
 - 空
 - 对于关键字、分界符、运算符，类别就可以表示其完整的信息
 - 符号表指针
 - 标识符
 - 常量表指针，特别是字符串常量
 - 节约存储空间
 - 理论上是程序员要去干的事情
 - Python程序中的常量就是有一个常量表

词法分析的术语

- 词元 - Token
 - 具有独立含义的最小语法单位，每个记号代表一类符号串
 - 例: <identifier>, <number>
- 模式 - Pattern
 - 描述某个记号的词素集合的构造规则
- 词素 - Lexeme
 - 源程序的符号序列，可匹配某个记号的模式
 - 如
 - 标识符: x, count, name, etc...

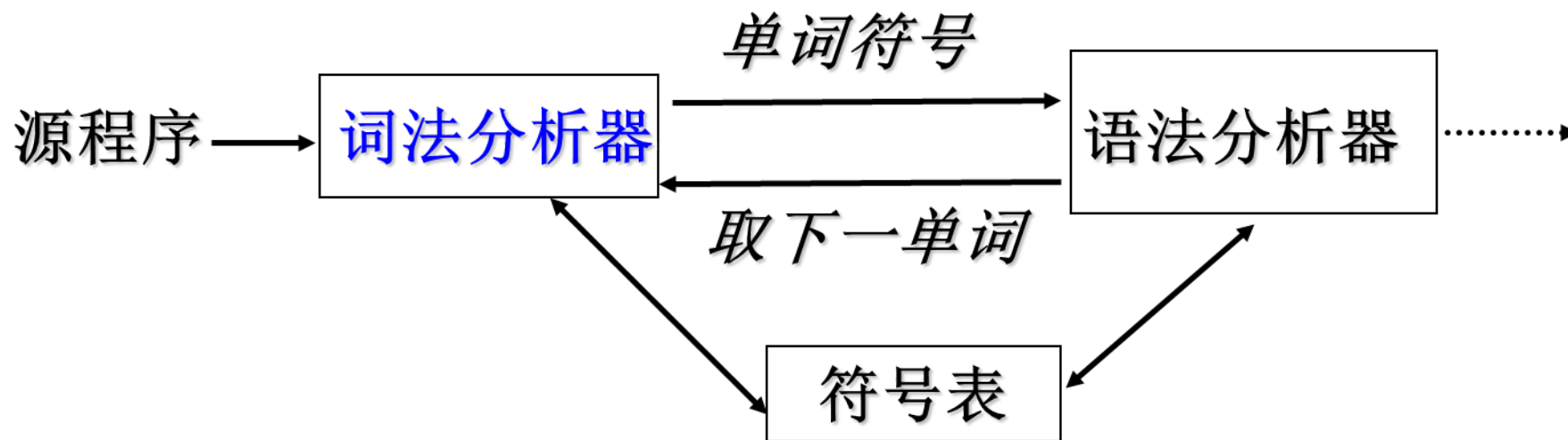
词法分析的设计形式

- 独立程序



词法分析的设计形式

- 语法分析器的子程序



独立词法分析阶段的优势

- 结构简洁、清晰和条理化，有利于简化编译器的设计
- 提高效率
- 增强可移植性

词法分析器的实现方法

- 使用程序设计语言手写
 - C、C++、Java、
 - 汇编
- 使用词法分析器生成器，从基于正则表达式的词法描述自动产生

手工实现

- 识别模式
 - 标识符识别
 - **字母开头**的字母数字串，后跟分界符或算符
 - 常数识别
 - 识别出算术常数并将其转变为二进制内码表示
 - 0x7F, 0777, 1ULL, 5E08, u”你好”
 - 算符和分界符的识别
 - 把多字符组成的算符和界符拼合成一个单词符号
 - =, ==, ++, --, >=, ->

手工实现

- 基于待识别字符串前缀区分识别模式
 - 数字开头->常数字面值
 - 0x前缀->十六进制字面值
 - 0前缀->八进制字面值
 - 其他正常数字
 - 小数点?
 - 0x7F.1F?
 - 规模（类型）后缀
 - 无符号Uu—unsigned
 - 长整型LI—long
 - 32位浮点后缀Ff—float
 - 64位浮点—无后缀
 - 80/96/128位浮点后缀L—long double

手工实现

- 基于待识别字符串前缀区分识别模式
 - 单引号/双引号——字符串or字符常量识别模式
 - 一般字符
 - 转义字符
 - `\t\n\r\\\"'`
 - `\uFFFF`
 - **`\EOL(End of Line)`**
 - 换行符——错误，一般来说，行内字符串不允许有换行，除非改行以\结尾
 - 三引号——跨行字符串识别模式
 - “/”符号——除法还是注释？
 - “//”行内注释
 - “/**/”跨行注释

手工实现

- 基于待识别字符串前缀区分识别模式
 - A-Za-z_作为首个字符
 - 关键字、标识符识别模式
 - 持续捕捉A-Za-z0-9_内的字符，直到一个不属于该范围内的字符出现
 - 运算符、空白字符等
 - 运算符
 - 大嘴法
 - 组合最近合法运算符，如==, +=, -=, ++, --, >=, <<, ...
 - 最小词素法
 - 仅识别**单字符**运算符，多字符运算符通过**语法分析**进行识别
 - 定义语法规则
 - equal: '=' '='
 - rightShift: Greater Greater;

手工实现

- 模式结束
 - 出现当前模式不能接受的情况，判断是否为词法错误，不是的话就返回匹配的串以及词元类型
- 错误处理
 - 出现当前语言的非法字符
 - ! @ # \$ % ^ & * `
 - 0d123
 - 整数常量的前缀错误还是分解成0 d123两个词素？
 - 0.toString()
 - 浮点数模式下的词法错误还是隐式转换后的成员函数调用？

词法的二义性

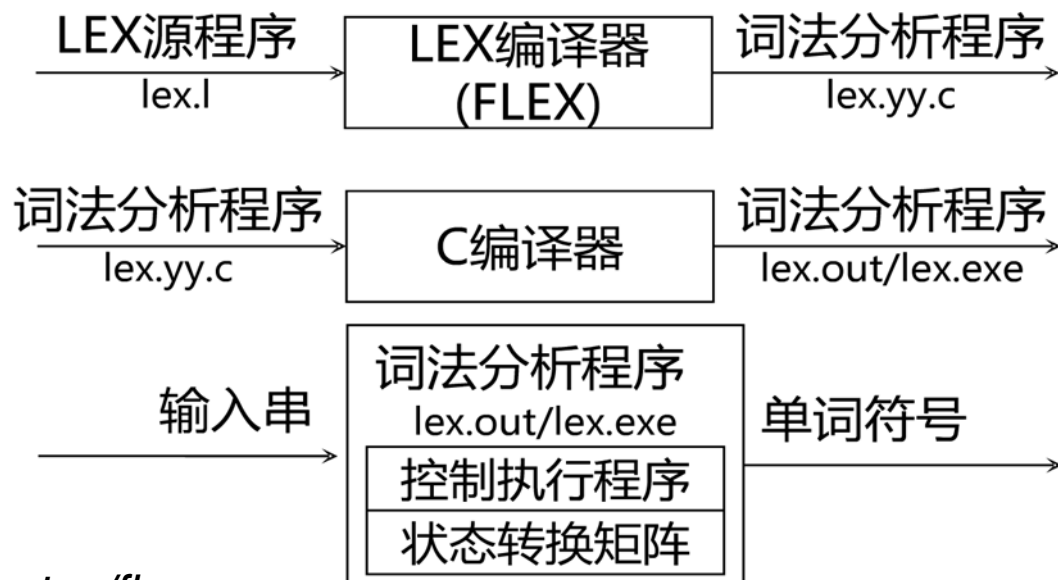
- 0x7F.1F
 - 64-bit Double类型的十六进制浮点数，整数部分为7F，小数部分为1F
 - 32-bit Float类型的十六进制浮点数，整数部分为7F，小数部分为1

词法分析器生成工具Lex

- 功能
 - 将输入的模式转换成一个状态转换图，生成代码，并存放到文件lex.yy.c中。
- Lex语言
 - Lex工具的输入表示方法

Lex的使用

- 用Lex语言编写输入文件，描述要生成的词法分析器
- 用Lex编译器(flex)将lex文件转换成C语言程序
- 编译C程序,即得词法分析器



Ref: <https://github.com/westes/flex>

一站式词法语法分析器生成工具ANTLR

- ANTLR(**A**Another **T**ool for **L**anguage **R**ecognition)
 - 一个强大的解析器生成器，用于读取、处理、执行或翻译结构化文本或二进制文件。
 - 它被广泛用于构建语言、工具和框架。
 - ANTLR 根据语法定义生成解析器，解析器可以构建和遍历解析树。
 - 所有编程语言的语法，都可以用ANTLR来定义。
 - ANTLR提供了大量的官方 grammar 示例，包含了各种常见语言，比如 Java、SQL、Javascript、PHP等等。

Ref: <https://juejin.cn/post/7018521754125467661>

基于ANTLR的四则混合运算计算器示例

作业

- 手工实现一个四则混合运算计算器的词法分析器
 - 限定C/C++语言
 - 自定义输出格式
 - 不使用现有的任何词法分析器生成工具（自己写的也可以）
 - 提交源代码和实验报告
 - 实验报告包含设计思路、主要分支目的及实现等
 - 用7z打包（zip压缩没强制字符集，可能会乱码）